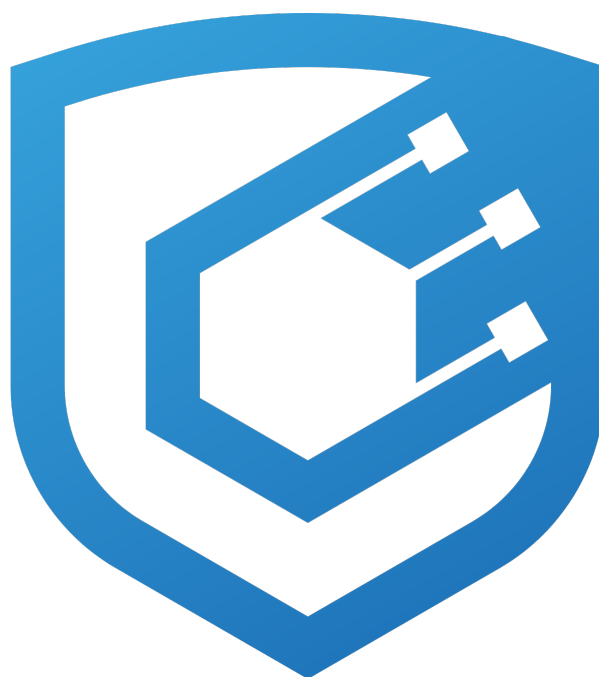




Securitize MatchHandler Audit Report

Prepared by [Cyfrin](#)

Version 2.0

Lead Auditors

[Kage](#)

[Immeas](#)

June 11, 2026

Contents

1	About Cyfrin	2
2	Disclaimer	2
3	Risk Classification	2
4	Protocol Summary	2
5	Audit Scope	2
6	Executive Summary	2
7	Findings	5
7.1	Low Risk	5
7.1.1	MatchHandler::matchOrder has no replay protection, allowing duplicate settlement	5
7.1.2	MatchHandler::matchOrder allows same seller and buyer	6
7.2	Informational	8
7.2.1	MatchHandler has several minor code-quality and documentation mismatches	8
7.2.2	Supplied MatchHandler deployment flow does not establish required registry authorization	8
7.3	Gas Optimization	10
7.3.1	Redundant custodialWallet storage read on the matchOrder settlement path	10
7.3.2	MatchHandler::addOperator, removeOperator call the public grantRole, revokeRole instead of the internal _grantRole, _revokeRole	10

1 About Cyfrin

Cyfrin is a Web3 security company dedicated to bringing industry-leading protection and education to our partners and their projects. Our goal is to create a safe, reliable, and transparent environment for everyone in Web3 and DeFi. Learn more about us at cyfrin.io.

2 Disclaimer

The Cyfrin team makes every effort to find as many vulnerabilities in the code as possible in the given time but holds no responsibility for the findings in this document. A security audit by the team does not endorse the underlying business or product. The audit was time-boxed and the review of the code was solely on the security aspects of the in-scope code being audited.

3 Risk Classification

	Impact: High	Impact: Medium	Impact: Low
Likelihood: High	Critical	High	Medium
Likelihood: Medium	High	Medium	Low
Likelihood: Low	Medium	Low	Low

4 Protocol Summary

`MatchHandler` is the on-chain settlement contract for Securitize's Alternative Trading System (ATS): an off-chain matching engine pairs buy and sell orders for DS Protocol security tokens, and the contract is designed to settle each matched trade in a single transaction by moving the DS Token from seller to buyer, paying the seller in a stablecoin net of fees, routing the combined seller and buyer fees to a custodial wallet, and registering the buyer in the DS Registry. It is deployed behind an ERC-1967 UUPS proxy with EIP-7201 namespaced storage, targets Ethereum, Arbitrum, and Avalanche C-Chain, and on its intended settlement path holds no balances of its own, transferring tokens directly between the trade parties via standing approvals. Two roles govern it: `DEFAULT_ADMIN_ROLE` controls upgrades, pausing, operator management, and the custodial-wallet address, while `OPERATOR_ROLE` (the trusted matching engine) is the sole caller of the settlement entry point. Trade eligibility, investor KYC/AML status, fee economics, and order authenticity are validated off-chain before settlement; the contract enforces structural guards only (non-zero addresses and amounts, a non-empty investor identifier, and a seller-fee ceiling).

5 Audit Scope

The audit scope was limited to:

```
contracts/ats/MatchHandler.sol
contracts/base/MatchHandlerStorage.sol
```

6 Executive Summary

Over the course of 2 days, the Cyfrin team conducted an audit on the [MatchHandler](#) code provided by [Securitize](#). The findings consist of 2 Low severity issues with the remainder being informational and gas optimizations.

Securitize's MatchHandler provides atomic on-chain settlement for security-token trades matched by its off-chain Alternate Trading System (ATS). The audit focused on the core settlement contract and its upgradeable storage implementation.

Cyfrin identified issues involving duplicate settlements, registry integration, counterparty validation, deployment configuration, and operational safeguards. All identified issues were addressed by the Securitize team, and fixes were fully verified by Cyfrin during the mitigation review.

The system retains centralization risk because privileged roles can execute settlements, pause operations, redirect fees, and upgrade the implementation. This risk can be reduced by securing administrative authority behind a multisignature wallet, separating operational and administrative roles.

Summary

Project Name	MatchHandler
Repository	bc-ats-sc
Commit	709be1e8839a...
Fix Commit	cf4db7a5e09a...
Audit Timeline	June 9th - June 10th, 2026
Methods	Manual Review

Issues Found

Critical Risk	0
High Risk	0
Medium Risk	0
Low Risk	2
Informational	2
Gas Optimizations	2
Total Issues	6

Summary of Findings

[L-1] MatchHandler::matchOrder has no replay protection, allowing duplicate settlement	Resolved
[L-2] MatchHandler::matchOrder allows same seller and buyer	Resolved
[I-1] MatchHandler has several minor code-quality and documentation mismatches	Partially Resolved
[I-2] Supplied MatchHandler deployment flow does not establish required registry authorization	Resolved
[G-1] Redundant custodialWallet storage read on the matchOrder settlement path	Resolved

[G-2] MatchHandler::addOperator, removeOperator call the public grant-Role, revokeRole instead of the internal _grantRole, _revokeRole	Resolved
--	----------

7 Findings

7.1 Low Risk

7.1.1 MatchHandler::matchOrder has no replay protection, allowing duplicate settlement

Description: MatchHandler::matchOrder receives no unique order identifier and stores no consumed-order state. Each authorized call independently executes all transfers, so an operator retry or duplicate queue delivery settles the same off-chain match again whenever both parties retain sufficient balances and aggregate allowances.

```
// contracts/ats/MatchHandler.sol:130
function matchOrder(...) external override whenNotPaused onlyRole(OPERATOR_ROLE) {
    ...
    _updateBuyerInRegistry(IDToken(dsToken), buyer);
    IDToken(dsToken).transferFrom(seller, buyer.wallet, dsTokenAmount);
    IERC20(stableCoin).safeTransferFrom(buyer.wallet, seller, sellerPayment);
    IERC20(stableCoin).safeTransferFrom(buyer.wallet, _getStorage().custodialWallet, totalFee);
    // @audit No order ID is marked consumed before or after settlement.
}
```

The registry update is idempotent for a consistent wallet and investor ID, so it does not prevent the repeated value transfers.

Impact: A duplicated operator submission can transfer the seller's DS Tokens and the buyer's stablecoin twice, and charge protocol fees twice, exceeding both users' authorized off-chain order amounts. It is noteworthy that this condition is limited to remaining balances and allowances, and only a trusted OPERATOR_ROLE can submit the duplicate.

Proof of Concept: Add the following test:

```
function test_POC_DuplicateMatchSettlesTwice() public {
    dsToken.mint(seller, DS_AMOUNT);
    vm.prank(seller);
    dsToken.approve(address(handler), DS_AMOUNT * 2);

    usdc.mint(buyer, USDC_AMOUNT + BUYER_FEE);
    vm.prank(buyer);
    usdc.approve(address(handler), (USDC_AMOUNT + BUYER_FEE) * 2);

    _matchOrder();
    _matchOrder();

    assertEq(dsToken.balanceOf(buyer), DS_AMOUNT * 2, "buyer receives the order twice");
    assertEq(usdc.balanceOf(seller), (USDC_AMOUNT - SELLER_FEE) * 2, "seller is paid twice");
    assertEq(usdc.balanceOf(custodial), (SELLER_FEE + BUYER_FEE) * 2, "fees are charged twice");
    assertEq(usdc.balanceOf(buyer), 0, "buyer pays for two settlements");
}
```

Recommended Mitigation: Consider adding a unique order or match ID to matchOrder, mark it consumed before external calls, and revert if it was already consumed.

```
if (settled[matchId]) revert MatchAlreadySettled(matchId);
settled[matchId] = true;
```

The off-chain engine should derive matchId deterministically from its canonical match record so retries remain idempotent.

Securitize: Fixed in commit [8ab63ad4](#)

Cyfrin: Verified.

7.1.2 MatchHandler::matchOrder allows same seller and buyer

Description: The audit scope states the following

Input validation rejects: zero addresses, zero amounts, sellerFee > stableCoinAmount , empty buyer.blockchainId , and seller == buyer.wallet. All reverts use typed custom errors from IMatchHandlerErrors.

MatchHandler::matchOrder does not enforce the requirement that seller != buyer.wallet.

```
// contracts/ats/MatchHandler.sol:130
function matchOrder(...) external override whenNotPaused onlyRole(OPERATOR_ROLE) {
    if (dsToken == address(0) || stableCoin == address(0) || seller == address(0)) revert ZeroAddress();
    if (buyer.wallet == address(0)) revert ZeroAddress();
    // @audit Missing: if (seller == buyer.wallet) revert InvalidCounterparties();
    ...
}
```

When both addresses are equal, the DS Token and seller-payment transfers are self-transfers and have no net economic effect. The combined fee still leaves the trader for the custodial wallet, and a normal Match event is emitted.

Impact: A malformed off-chain match is recorded on-chain as genuine trading volume even though no assets change counterparties, while the user is charged sellerFee + buyerFee. It is noted however that exploitation requires an erroneous or compromised trusted operator.

Proof of Concept: Run the following test

```
function test_POC_SelfMatchProducesFeeOnlySettlement() public {
    address trader = makeAddr("trader");
    uint256 traderFunds = USDC_AMOUNT + BUYER_FEE;

    dsToken.mint(trader, DS_AMOUNT);
    vm.prank(trader);
    dsToken.approve(address(handler), DS_AMOUNT);

    usdc.mint(trader, traderFunds);
    vm.prank(trader);
    usdc.approve(address(handler), traderFunds);

    IMatchHandler.Investor memory selfBuyer = _makeBuyer();
    selfBuyer.wallet = trader;

    vm.prank(operator);
    handler.matchOrder(
        address(dsToken),
        address(usdc),
        trader,
        DS_AMOUNT,
        USDC_AMOUNT,
        SELLER_FEE,
        BUYER_FEE,
        selfBuyer
    );

    assertEq(dsToken.balanceOf(trader), DS_AMOUNT, "DS token transfer nets to zero");
    assertEq(usdc.balanceOf(trader), USDC_AMOUNT - SELLER_FEE, "only combined fees leave trader");
    assertEq(usdc.balanceOf(custodial), SELLER_FEE + BUYER_FEE, "custodial receives fees");
}
```

Recommended Mitigation: Enforce distinct counterparties in matchOrder:

```
if (seller == buyer.wallet) revert InvalidCounterparties();
```

Securitize: Fixed in commit <https://github.com/securitize-io/bc-ats-sc/commit/36a9bfda57af4b23523717b781347717a>

Cyfrin: Verified.

7.2 Informational

7.2.1 MatchHandler has several minor code-quality and documentation mismatches

Description: A set of low-risk inconsistencies and omissions:

1. *Attribute arrays are forwarded without length validation* MatchHandler::_updateBuyerInRegistry passes the buyer's three attribute arrays straight to updateInvestor, which requires them to be equal length; a malformed Investor reverts deep in the registry rather than at the contract boundary.

```
// contracts/ats/MatchHandler.sol:247
registry.updateInvestor(buyer.blockchainId, "", buyer.country, wallets,
    buyer.investorAttributeIds, buyer.investorAttributeValues, buyer.investorAttributeExpirations); //
    ↳ @audit no local length check
// external/dstoken/contracts/registry/RegistryService.sol:69
require(_attributeValues.length == _attributeIds.length, "Wrong length of parameters");
```

2. **Investor NatSpec documents a field that is intentionally absent.** The NatSpec for IMatchHandler.Investor documents a collisionHash parameter, but the struct has no such field (contracts/interfaces/IMatchHandler.sol:36,43-50). This is a documentation mismatch : doc/architecture.md:86-90 explicitly states that callers do not need to provide the registry's collisionHash argument and that _updateBuyerInRegistry intentionally passes "" (MatchHandler.sol:249). DS Protocol v4.1.0 only stores that value when registering a new investor and does not validate it or use it in an on-chain security check.

3. **Role naming drift.** The dev brief and audit-scope PDF refer to the settlement role as OWNER_ROLE; the code defines and uses OPERATOR_ROLE (MatchHandler.sol:79).

4. **No CustodialWalletSet event on initialization.** initialize sets custodialWallet without emitting CustodialWalletSet (MatchHandler.sol:115), while setCustodialWallet does (:192). Indexers tracking only that event miss the initial value.

Recommended Mitigation:

- Validate investorAttributeIds.length == investorAttributeValues.length == investorAttributeExpirations.length in matchOrder before the registry call.
- Remove the stale @param collisionHash entry from the Investor NatSpec, or clarify nearby documentation that the registry argument is intentionally supplied internally as "". Do not add an ABI field unless the intended integration changes and operators are expected to provide this value.
- Align documentation to use OPERATOR_ROLE.
- Emit CustodialWalletSet(address(0), custodialWallet_) in initialize.

Securitize:

1. Acknowledged.
2. Fixed in commit [dc4ca3d](#)
3. Readme and project documentation and natSpec were updated
4. Acknowledged. Typically we never emit events during storage initialization, only for updates

Cyfrin: 2. Verified.

7.2.2 Supplied MatchHandler deployment flow does not establish required registry authorization

Description: MatchHandler::_updateBuyerInRegistry calls RegistryService::updateInvestor from the proxy address on every settlement. In DS Protocol v4.1.0, that function is protected by onlyExchangeOrAbove, so the proxy must hold EXCHANGE, ISSUER, TRANSFER_AGENT, or MASTER in each token's Trust Service.

```
// contracts/ats/MatchHandler.sol:247
registry.updateInvestor(...); // @audit msg.sender is the MatchHandler proxy
```

```
// dstoken/contracts/registry/RegistryService.sol:60
function updateInvestor(...) public override onlyExchangeOrAbove returns (bool) {
```

This authorization does not exist in the current MatchHandler deployment flow:

- ignition/modules/MatchHandler.ts:30-53 only deploys the implementation and proxy and initializes the proxy with the MatchHandler admin, operator, and custodial-wallet addresses. It does not interact with a DS Token's Trust Service.
- tasks/actions/verify-deployment.ts:15-56 verifies the proxy version, custodial wallet, pause state, and internal MatchHandler role holders, but does not accept a DS Token address or query its Trust Service.

Impact: A deployment that follows the documented procedure can be initialized successfully but have every matchOrder call revert for a listed DS Token until the external role is granted.

Recommended Mitigation: Consider adding a mandatory listing/deployment step that grants the proxy EXCHANGE-or-above for every supported DS Token, and extend deployment verification to query the Trust Service role.

Also, document that revoking this external role pauses settlement for the affected token.

Securitize: Fixed in commit [cf4db7a](#)

Cyfrin: Verified.

7.3 Gas Optimization

7.3.1 Redundant `custodialWallet` storage read on the `matchOrder` settlement path

Description: On the success path of `MatchHandler::matchOrder`, the namespaced storage field `custodialWallet` is loaded from storage twice across separate statements - once to address the fee transfer (only when `totalFee > 0`) and again as the final `emit` argument. The optimizer cannot dedupe these because they span statement boundaries (and an intervening external `safeTransferFrom`). The first read occurs only inside the `totalFee > 0` branch, which also warms the slot, so on a settlement that charges a fee the second read is a redundant warm read (roughly 100 gas saved by caching); when `totalFee` is zero only the event read runs and there is no redundancy. The saving therefore applies per fee-charging settlement on this hot path.

```
contracts/ats/MatchHandler.sol
162:         IERC20(stableCoin).safeTransferFrom(buyer.wallet, _getStorage().custodialWallet,
↪ totalFee);
165:         emit Match(seller, buyer.wallet, dsToken, stableCoin, dsTokenAmount, stableCoinAmount,
↪ sellerFee, buyerFee, _getStorage().custodialWallet);
```

Recommended Mitigation: Read the field once near the top of the fee block and reuse the local in both the transfer and the event:

```
address custodial = _getStorage().custodialWallet;
uint256 totalFee = sellerFee + buyerFee;
if (totalFee > 0) {
    IERC20(stableCoin).safeTransferFrom(buyer.wallet, custodial, totalFee);
}
emit Match(seller, buyer.wallet, dsToken, stableCoin, dsTokenAmount, stableCoinAmount, sellerFee,
↪ buyerFee, custodial);
```

Securitize: Fixed in commit [ffe8e33](#)

Cyfrin: Verified.

7.3.2 `MatchHandler::addOperator`, `removeOperator` call the public `grantRole`, `revokeRole` instead of the internal `_grantRole`, `_revokeRole`

Description: `MatchHandler::addOperator` and `MatchHandler::removeOperator` are already gated by `onlyRole(DEFAULT_ADMIN_ROLE)`, yet they call OpenZeppelin's public `grantRole`, `revokeRole` (`contracts/ats/MatchHandler.sol:175` and `contracts/ats/MatchHandler.sol:180`). Those public functions carry their own `onlyRole(getRoleAdmin(OPERATOR_ROLE))` modifier; since `OPERATOR_ROLE` is administered by `DEFAULT_ADMIN_ROLE` (the OpenZeppelin default), that built-in check re-verifies the caller's `DEFAULT_ADMIN_ROLE` membership that the wrapper's own modifier already checked, costing a redundant warm SLOAD (and the surrounding `hasRole` plumbing) on every call. The contract's own `initialize` already uses the internal `_grantRole` (`contracts/ats/MatchHandler.sol:112-113`), so the two operator setters are inconsistent with it.

```
contracts/ats/MatchHandler.sol
175:         grantRole(OPERATOR_ROLE, operator);
180:         revokeRole(OPERATOR_ROLE, operator);
```

Recommended Mitigation: Call the internal variants, which perform the role write without the redundant access check (the `onlyRole(DEFAULT_ADMIN_ROLE)` wrapper already authorizes the caller, matching what `initialize` does):

```
_grantRole(OPERATOR_ROLE, operator);    // addOperator
_revokeRole(OPERATOR_ROLE, operator);    // removeOperator
```

This is an admin-only, infrequently-called path, so the per-call saving is small; the change also restores consistency with `initialize`.

Securitize: Fixed in commit [a54fa5d](#)

Cyfrin: Verified.